A Project Report

on

MEDTWIN: REALISTIC AND PRIVATE SYNTHETIC PATIENT DATA WITH MISSING VALUES

carried out as part of the ***Project (Subject Code)*** *Submitted*

by

Pranav Ladha (23FE10CDS00455)
Devisha Jain (23FE10CDS00493)
Nandini Nahata (23FE10CDS00376)
Jiya Gakhar (23FE10CDS00480)

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY (B.Tech)

in

Department of Data Science and Engineering



School of Computer Science and Engineering,

MANIPAL UNIVERSITY JAIPUR
RAJASTHAN, INDIA

December 2026

Date: _____

CERTIFICATE

This is to certify that the project entitled **"MedTwin: Realistic and Private Synthetic Patient Data with Missing Values"** has been carried out by Pranav Ladha (23FE10CDS00455), Devisha Jain (23FE10CDS00493), Nandini Nahata (23FE10CDS00376) and Jiya Gakhar (23FE10CDS00480), students of the Department of Data Science and Engineering, Manipal University Jaipur, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology**. The work presented in this report is a bonafide record of the work done by them under my guidance and supervision during [start month, year] to [end month, year].

Dr. Chirag Joshi
Assistant Professor (Selection Grade)
Department of Data Science & Engineering
Manipal University Jaipur

ACKNOWLEDGMENTS

We thank our project guide, **Dr. Chirag Joshi**, Assistant Professor (Selection Grade), Department of Data Science & Engineering, Manipal University Jaipur, for guiding us through this project. He pushed us to evaluate synthetic data fairly and to test every claim we made, and the project is much better for it.

We also thank the Head of the Department and the faculty of the Department of Data Science and Engineering for their support, and the School of Computer Science and Engineering and Manipal University Jaipur for letting us take up this work as part of our B.Tech programme.

The UCI Heart Disease and Pima Indians Diabetes datasets are public. We are grateful to the people who collected and shared them, and to the authors of the open-source libraries we used.

Parts of the code, analysis and text of this project were developed with the help of an AI assistant (Claude, Anthropic). All results were produced by running the project's code, and we have reviewed and verified the code, results and text and take full responsibility for them.

Finally, we thank our families and friends for their support.

Pranav Ladha
Devisha Jain
Nandini Nahata
Jiya Gakhar

ABSTRACT

Medical machine-learning models need patient data, but privacy laws and hospital rules limit who may see it. Synthetic data is one answer. A generative model learns the patterns in real patient records and then produces artificial records that follow the same patterns. Real clinical data has gaps, and they are not random: whether a test was done often depends on the hospital. Most synthetic-data tools fill these gaps before training, so their output has no gaps at all. Records like that do not look like real hospital data and are easy to tell apart from it.

This project presents **MedTwin**, a conditional Wasserstein generative adversarial network with gradient penalty (WGAN-GP) that keeps "not measured" as part of each record. For every column that can be missing, MedTwin learns a yes/no flag that says whether the value was measured. A second flag handles values that are exactly zero, and each synthetic patient is generated for a chosen hospital, so the model picks up each hospital's testing habits. We also propose two ways to evaluate missing values. Missing-value fidelity scores check whether the gaps in synthetic data are realistic per column, per hospital and as whole patterns. A missing-value privacy audit checks whether keeping realistic gaps makes patients easier to identify.

We compared MedTwin with CTGAN, TVAE and TabDDPM on the four-hospital UCI Heart Disease data (920 patients) and the Pima Indians Diabetes data (768 patients). Each comparison was repeated five times and always tested on real patients that no model had seen. On heart disease, a classifier could tell MedTwin's patients from real ones 61.5% of the time, against 74.9–92.8% for the other methods (50% would mean they cannot be told apart at all). A disease predictor trained only on MedTwin's data kept 97.5% of the performance of one trained on real data, level with the best other method. MedTwin's missing-value patterns were almost as close to the real data as a second sample of real patients (pattern distance 0.174 against 0.152), and no other method put the gaps at the right hospitals. Privacy attacks found no sign that MedTwin memorised its training patients, while TabDDPM did memorise them on the diabetes data. We also found that keeping realistic gaps has a small privacy cost for patients whose gaps are unusual.

Keywords: synthetic data, generative adversarial networks, missing data, electronic health records, privacy evaluation, deep learning

LIST OF FIGURES

Figure No	Figure Title	Page No
Figure 3.1	Block diagram of the MedTwin pipeline	12
Figure 3.2	Architecture of the MedTwin conditional WGAN-GP	13
Figure 3.3	Evaluation framework	13
Figure 4.1	MedTwin interactive dashboard (heart disease data)	16
Figure 5.1	Heart disease comparison of the four methods (mean ± SD over five seeds; dashed lines show the ideal value)	18
Figure 5.2	Pima diabetes comparison of the four methods	19
Figure 5.3	Share of heart disease patients with each value not measured, per hospital: real (blue) and MedTwin (orange)	21
Figure 5.4	Distributions of the number columns: real (blue) vs MedTwin (orange), measured values only	23
Figure 5.5	Correlations between number columns: real, MedTwin and their difference	23

LIST OF TABLES

Table No	Table Title	Page No
Table 2.1	Technologies and tools used	5
Table 2.2	Comparison with recent related work	6
Table 3.1	Summary of the datasets	8
Table 3.2	Heart disease columns	8
Table 3.3	Share of heart disease patients with each value not measured, by hospital	9
Table 3.4	MedTwin model architecture	10
Table 3.5	Training settings	10
Table 3.6	Evaluation measures	11
Table 4.1	Software modules	14
Table 4.2	Methods compared and average training time (heart disease)	15
Table 5.1	Heart disease results (mean ± SD over five seeds)	18
Table 5.2	Pima diabetes results (mean ± SD over five seeds)	19
Table 5.3	Missing-value fidelity (mean ± SD; lower is better)	19
Table 5.4	Missing-value privacy audit, heart disease (mean ± SD; 0.5 or 0 is ideal)	21
Table 5.5	Ablation study (mean ± SD; * = clear difference from the full model)	22
Table 5.6	Sensitivity of MedTwin to its main settings, heart disease (* = clear difference from the default)	22
Table A.1	Sample of MedTwin's synthetic heart disease patients	30

Contents

	Title	Page No
	Acknowledgement	i
	Abstract	ii
	List Of Figures	iii
	List Of Tables	iv
	List of Abbreviations	vi
Chapter 1	**INTRODUCTION**	1
1.1	Introduction to Work Done / Motivation (Overview, Applications & Advantages)	1
1.2	Project Statement / Objectives of the Project	2
1.3	Organization of Report	3
Chapter 2	**BACKGROUND MATERIAL**	4
2.1	Conceptual Overview (Concepts / Theory Used)	4
2.2	Technologies Involved	5
2.3	Literature Survey	6
2.4	Research Gap	7
Chapter 3	**METHODOLOGY**	8
3.1	Detailed Methodology that will be Adopted	8
3.2	Circuit Layouts / Block Diagrams	12
Chapter 4	**IMPLEMENTATION**	14
4.1	Modules	14
4.2	Prototype	15
4.3	Testing	16
Chapter 5	**RESULTS AND ANALYSIS**	18
5.1	Heart Disease: MedTwin Compared with Other Methods	18
5.2	Diabetes: The Same Picture	19
5.3	Are the Missing Values Realistic?	19
5.4	Does Keeping Missing Values Cost Privacy?	21
5.5	What Each Component Contributes (Ablation)	22
5.6	Sensitivity to Settings	22
5.7	Quality of the Generated Values	23
5.8	Challenges Faced and Learnings	23
Last Chapter	**CONCLUSIONS & FUTURE SCOPE**	25
6.1	Conclusions	25
6.2	Future Scope of Work	26
	REFERENCES	27
	ANNEXURES	30

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
AUC	Area Under the (ROC) Curve
CSV	Comma-Separated Values (file format)
CTGAN	Conditional Tabular Generative Adversarial Network
DCR	Distance to Closest Record
EHR	Electronic Health Record
GAN	Generative Adversarial Network
MIA	Membership Inference Attack
ML	Machine Learning
MUJ	Manipal University Jaipur
RF	Random Forest
SD	Standard Deviation
TabDDPM	Tabular Denoising Diffusion Probabilistic Model
TRTR	Train on Real, Test on Real
TSTR	Train on Synthetic, Test on Real
TVAE	Tabular Variational Autoencoder
TVD	Total Variation Distance
UCI	University of California, Irvine (Machine Learning Repository)
VA	Veterans Affairs (VA Long Beach hospital)
WGAN-GP	Wasserstein GAN with Gradient Penalty
YAML	YAML Ain't Markup Language (settings file format)

INTRODUCTION

1.1) Introduction to Work Done / Motivation (Overview, Applications & Advantages)

Overview

Hospitals now use machine learning to predict disease and to help doctors with diagnosis and treatment. Building these models takes a lot of real patient data. That data is sensitive, so laws such as HIPAA in the USA and GDPR in Europe, along with hospital ethics rules, limit who can see it. Students, researchers and small organisations often cannot get access, and some useful projects never start.

Synthetic data is one way around this [1, 2]. A generative model learns the statistical patterns of real patient records and then produces new records that follow those patterns but belong to no real person. If the synthetic records look real, work well for training models and do not give away the real patients, they can be shared in place of the real data, for example for research, teaching or software testing.

In this project we built **MedTwin**, a model that creates synthetic patient records from real hospital data. MedTwin is a conditional Wasserstein generative adversarial network with gradient penalty (WGAN-GP) [3, 4, 5]. We tested it against three widely used synthetic-data methods on two public datasets: the UCI Heart Disease data from four hospitals [6, 7] and the Pima Indians Diabetes data [8].

Motivation

Real clinical tables have gaps, and the gaps follow a pattern. Whether a test was done depends on the hospital, the doctor and the patient. Some examples from the heart disease data used here:

- every patient from Switzerland has a cholesterol value of 0, which really means "not measured" (100% missing);
- the number of major blood vessels seen on a scan is missing for 96–99% of patients at three of the four hospitals, but almost never at Cleveland;
- resting blood pressure is missing for 28% of patients at VA Long Beach but for almost nobody elsewhere.

Most synthetic-data tools cannot handle missing values, so people usually fill each gap with a typical value such as the median before training. The synthetic data then has no gaps at all, unlike real hospital data, and the filled-in values were never measured on anyone. A synthetic copy of a multi-hospital dataset should reproduce *which* tests were done *where*. That is the starting point of this project.

Applications

- Hospitals could share synthetic versions of their data with researchers without exposing real patients.
- Students could practise building medical models on data that has real-world problems such as missing values.
- Developers could test data pipelines and prediction models on data that behaves like hospital data, gaps included.
- Because MedTwin generates patients for a chosen hospital, one model can represent the testing habits of several hospitals.
- The evaluation tools from this project can be used to compare any synthetic-data generator, including on how well it handles missing values.

Advantages

- Synthetic records do not belong to real patients, so sharing them carries less privacy risk.
- Missing values are generated as gaps instead of being filled with made-up numbers, so the data keeps the structure of real records.
- A new dataset only needs a new settings file. The code stays the same.
- Every result is measured on real patients the model never saw, repeated with five seeds and compared with paired statistics.

1.2) Project Statement / Objectives of the Project

Problem statement. Build and test a model that generates synthetic patient records from incomplete clinical data collected at several hospitals, keeping the real pattern of missing values. The records should look real, be useful for training prediction models and protect the real patients. We also need ways to measure how realistic the missing values in synthetic data are, and whether keeping them puts patients at risk.

The main objectives of the project are:

1. Build a generator (MedTwin) that learns which values were measured, generates each patient for a specific hospital and reproduces values that are exactly zero.
2. Make the system work for any table of medical data through a settings file, and show this on two datasets.
3. Test on real patients held out from training, with five repeated runs and paired statistical tests.
4. Compare MedTwin with CTGAN, TVAE and TabDDPM under the same conditions.
5. Define scores that measure how realistic the gaps in synthetic data are, per column, per hospital and as whole patterns.
6. Test whether keeping realistic gaps makes patients easier to identify.

7. Measure what each part of the model adds by switching parts off one at a time, and check how much the results depend on the main settings.

1.3) Organization of Report

The rest of this report is laid out as follows.

1. Chapter 1 introduces the problem, its applications and the objectives.
2. Chapter 2 covers the background (synthetic data, GANs, missing data, evaluation and privacy), the tools we used and related work.
3. Chapter 3 describes the method: datasets, data preparation, model design, training, the test protocol and the new missing-value measures, with block diagrams.
4. Chapter 4 describes the software modules, the dashboard prototype and the automated tests.
5. Chapter 5 presents the results on both datasets, the missing-value scores, the privacy audit, the ablation and sensitivity studies, and the problems we ran into.
6. Chapter 6 gives the conclusions and ideas for future work.
7. The references (IEEE style) and annexures come last. The annexures hold sample synthetic records, code snippets and instructions for running the project.

BACKGROUND MATERIAL

2.1) Conceptual Overview (Concepts / Theory Used)

1. Synthetic data

Synthetic data is artificial data made by a model that has learned the patterns of real data. Reviews of the field [1, 19] judge it on realism, usefulness and privacy. Realistic data looks like the real data, both column by column and in how the columns relate. Useful data trains models that work on real patients. Private data does not reveal which real people were used to make it.

2. Generative Adversarial Networks (GANs)

A GAN [3] has two neural networks that are trained against each other. The generator turns random noise into fake records, and the discriminator tries to tell the fake records from real ones. Over training, the generator's records get harder and harder to tell apart from real data.

3. Wasserstein GAN with Gradient Penalty (WGAN-GP)

Standard GANs are often unstable and can end up producing only a few kinds of record. The Wasserstein GAN [4] replaces the discriminator's real/fake decision with a critic that gives each record a score, and these scores estimate the Wasserstein distance between real and fake data. The gradient-penalty version [5] penalises large gradients in the critic, which makes training much more stable. The critic loss is:

$$L_D = \mathrm{E}[D(\text{fake}, c)] - \mathrm{E}[D(\text{real}, c)] + \lambda \cdot \mathrm{E}[(\|\nabla D(\text{mixed}, c)\| - 1)^2], \quad \text{with } \lambda = 10.$$

4. Conditional generation

In a conditional GAN, both networks also receive some extra information, called the condition. In MedTwin the condition is the hospital. The generator creates a patient *for* a given hospital, and the critic judges whether the patient is realistic *for that hospital*. This is how the model learns which tests each hospital does.

5. Gumbel-softmax for categories

Categorical columns (such as chest pain type) must take exactly one value. The Gumbel-softmax technique [20] lets a neural network produce a smooth, trainable approximation of a one-hot choice during training and an exact one-hot choice when sampling.

6. Missing data

Values go missing for many reasons. In data from several hospitals, the main reason is usually local practice: some hospitals just do not run a given test. Common tools either fill missing values with a typical value, which is called

imputation [21], or learn a separate model of which values are missing [22]. If imputation is used before generating synthetic data, the gaps disappear and the synthetic records no longer show which tests were done.

7. Evaluating synthetic data

- Fidelity measures how closely the distribution of each column matches the real data. We measure it as histogram overlap.
- Detectability is the accuracy of a classifier trained to tell real records from synthetic ones. At 50% the two cannot be told apart. It measures realism and says nothing about privacy [23].
- TSTR (Train on Synthetic, Test on Real) [24] trains a disease predictor on synthetic data and tests it on real patients. Its AUC is compared with the same predictor trained on real data (TRTR).
- Distance to Closest Record (DCR) checks whether synthetic records sit suspiciously close to the patients used for training.
- A Membership Inference Attack (MIA) [25, 26] tries to tell whether a given real patient was in the training data. An AUC of 0.5 means the attack does no better than guessing.

Passing these attacks is evidence of privacy, but it is not a guarantee [27].

8. Statistical testing and ablation

Results change with random choices, so each experiment is repeated with five seeds. Methods are compared seed by seed with a paired t-based 95% confidence interval, and we call a difference *clear* only when the interval does not include zero. An ablation study switches off one part of the system at a time to see what that part adds.

2.2) Technologies Involved

All the code is written in Python with open-source libraries. Table 2.1 lists the main ones and what we used them for.

Table 2.1: Technologies and tools used

Area	Technology / version	Used for
Programming language	Python 3.9	All code
Deep learning	PyTorch 2.8.0 [9]	Generator, critic and training
Data handling	pandas 2.3.3, NumPy 2.0.2	Loading, cleaning and encoding data
Machine learning	scikit-learn 1.6.1 [10]	Random forests, nearest neighbours, AUC, scaling
Comparison methods	ctgan 0.12.1 (CTGAN, TVAE); TabDDPM written in this project	Baselines

Area	Technology / version	Used for
Statistics	SciPy	Paired t-tests and confidence intervals
Visualisation	matplotlib 3.9.4, seaborn 0.13.2	Figures
Dashboard	Streamlit 1.50.0	Interactive web prototype
Configuration	PyYAML 6.0.3	Dataset settings files
Testing	pytest	31 automated tests
Version control	Git	History of all changes
Documents	LaTeX (Springer LNCS), Tectonic	Research paper

Random forests [28] were used for the detectability, utility and privacy tests, and the Adam optimiser [29] for training the networks. The baselines CTGAN and TVAE [30] come from the open-source ctgan library, and TabDDPM [31] was re-implemented in this project following its published design.

2.3) Literature Survey

Early work on synthetic health records used GANs for binary and count features (medGAN [32]). CTGAN [30] introduced a conditional GAN for mixed tables of numbers and categories, and the same paper proposed TVAE, a variational autoencoder for tables. CTAB-GAN [33] improved the handling of mixed and long-tailed variables. Diffusion models such as TabDDPM [31] and TabSyn [34] now top many tabular benchmarks. EHR-Safe [35] was built for the messy structure of real electronic health records. Large benchmark studies [2, 19, 36] compare many generators on realism, usefulness and privacy, and recent work shows that tabular diffusion models can memorise their training records [37].

Several recent studies generated synthetic versions of the same or similar data, and Table 2.2 compares them with this project. Their numbers cannot be compared directly with ours, because each study uses different data, splits and measures.

Table 2.2: Comparison with recent related work

Work	What they did	Their result	How this project compares
Wan Abu Bakar et al., 2025 [11]	CTGAN on Cleveland heart data (303 patients); gaps filled with the most common value; single run	56% accuracy training on synthetic, testing on real (5-class)	We use all 4 hospitals, unseen test patients and 5 repeats; MedTwin clearly beats our own CTGAN baseline
Subedi et al., 2025 [12]	CTGAN on Pima diabetes	Random forest accuracy 82% (real) → 63% (synthetic)	MedTwin keeps 98% of real-data performance (AUC ratio)

Work	What they did	Their result	How this project compares
Alqulaity & Yang, 2024 [13]	Enhanced conditional GAN, 218 cardiovascular patients, one hospital	Distribution statistics only; no utility or privacy test	We test realism, usefulness and privacy on unseen patients
Lazzaro et al., 2026 [14]	Bayesian networks on UCI heart sites, trained on Cleveland only	Data augmentation results	We model all 4 hospitals, including the very incomplete ones
HealthGen [15], MissDiff [16], Wang et al. [17], IGAMT [18]	Generators that keep missing values (time series, diffusion, plug-in, private EHR)	Show that keeping missingness matters	Same goal; we add hospital-conditioned gaps, scores for gap realism and the first audit of their privacy cost

2.4) Research Gap

The survey showed these open problems:

1. Studies on the UCI heart and Pima data usually fill missing values, use a single run, test on patients the generator saw, and do not measure privacy.
2. Methods that keep missing values exist, but there is no standard way to measure whether the generated gaps are realistic, particularly per hospital.
3. No study has tested whether keeping realistic missing values creates a privacy risk for patients with unusual gaps.

METHODOLOGY

3.1) Detailed Methodology that will be Adopted

The project was carried out in eight stages, summarised in the block diagram of Fig. 3.1 (Section 3.2). Each stage is described below.

Step 1: Data collection and understanding

We used two public datasets. The UCI Heart Disease data [6, 7] combines patients from four hospitals, and the Pima Indians Diabetes data [8] was taken from OpenML (dataset 37) [38]. Table 3.1 summarises both.

Table 3.1: Summary of the datasets

	Heart disease (UCI)	**Pima diabetes**
Patients	920	768
Columns used	7 numbers + 7 categories + hospital	9 numbers
Hospitals	Cleveland (304), Hungary (293), VA Long Beach (200), Switzerland (123)	1 (one clinic)
Prediction target	heart disease (severity > 0): 55.3% yes	diabetes: 34.9% yes
Missing values	many; depend strongly on the hospital	zeros that mean "not measured"

Table 3.2 lists the heart disease columns and how often each is missing. Table 3.3 shows that the missing values depend strongly on the hospital. A realistic synthetic dataset has to reproduce this.

Table 3.2: Heart disease columns

Column	**Meaning**	**Type**	**Missing**
age	Age in years	number	0.0%
sex	Sex	category	0.0%
dataset	Hospital (site)	site	0.0%
cp	Chest pain type	category	0.0%
trestbps	Resting blood pressure (mmHg)	number	6.5%
chol	Serum cholesterol (mg/dl)	number	22.0%
fbs	Fasting blood sugar > 120 mg/dl	category	9.8%
restecg	Resting ECG result	category	0.2%
thalch	Maximum heart rate (bpm)	number	6.0%
exang	Chest pain during exercise	category	6.0%
oldpeak	ST depression on ECG	number	6.7%
slope	Slope of ST segment	category	33.6%

Column	Meaning	Type	Missing
ca	Major vessels seen on scan (0–3)	number	66.4%
thal	Thalassemia test result	category	52.8%
num	Disease severity (0 = none, 1–4)	number	0.0%

Table 3.3: Share of heart disease patients with each value not measured, by hospital

Hospital (patients)	chol	trestbps	ca	thal	slope	fbs
Cleveland (304)	0%	0%	2%	1%	0%	0%
Hungary (293)	8%	0%	99%	90%	65%	3%
Switzerland (123)	100%	2%	96%	42%	14%	61%
VA Long Beach (200)	28%	28%	99%	83%	51%	4%

For the diabetes data, a value of 0 for glucose, blood pressure, skin fold, insulin or BMI is physically impossible and means "not measured" (insulin is missing for 48.7% of patients). Zero pregnancies, however, is a real value. These rules are written in each dataset's settings file.

Step 2: Train / test split

For each experiment, 20% of patients are held out and never shown to any model. All usefulness and privacy tests are measured against these unseen patients. The split is stratified by the outcome so both parts have the same share of sick patients.

Step 3: Encoding records with their missing values

- Zeros that mean missing (listed in the settings file) are turned into blanks.
- Every number column that is sometimes missing gets a 0/1 flag that says whether it was measured. The blank itself is replaced by the median, which is only a placeholder because the flag marks it as meaningless.
- 40% of heart patients have an ST depression of exactly zero. A separate 0/1 zero flag lets the network reproduce this spike.
- Numbers are scaled to the range 0–1, and each category column is one-hot encoded with "not measured" as an extra category.
- The hospital is not generated. It is given to the model as a one-hot condition.

Step 4: Model design and training

MedTwin is a conditional WGAN-GP. Its architecture is given in Table 3.4 and drawn in Fig. 3.2. In addition to the Wasserstein loss and gradient penalty, the generator is penalised when the correlations between columns in its output

differ from those in the real batch (correlation loss, weight $\alpha = 2$). Columns that are constant within a batch are skipped, because their correlation is undefined and would otherwise break training. Training settings are listed in Table 3.5.

Table 3.4: MedTwin model architecture

Part	Specification
Generator input	Random noise (128 numbers) + one-hot hospital condition
Generator layers	Linear 256 → LeakyReLU(0.2) → BatchNorm → Linear 512 → LeakyReLU → BatchNorm → Linear (output)
Generator outputs	Sigmoid for numbers and flags (0–1); Gumbel-softmax ($\tau = 0.2$) per categorical column
Critic layers	Linear 512 → LeakyReLU → Dropout 0.3 → Linear 256 → LeakyReLU → Dropout 0.3 → Linear 1
Critic input	Record + hospital condition, so realism is judged per hospital
Loss	Wasserstein loss + gradient penalty ($\lambda = 10$) + correlation penalty ($\alpha = 2$)

Table 3.5: Training settings

Setting	Value
Optimiser	Adam [29], learning rate 0.0001, betas (0.0, 0.9)
Epochs / batch size	600 epochs, batch size 64
Critic updates	5 critic updates per generator update
Records generated	2000 per run
Train / test split	80% / 20%, stratified by outcome
Repeats	5 seeds (0–4); each seed changes the split and the model's starting point

Step 5: Generation and decoding

To create synthetic data, a hospital is drawn for each patient in the same proportions as the real data, and the generator produces 2000 patients. Values are converted back to real units. If a patient's zero flag is set, the value becomes 0; if its measurement flag is set, the value is left blank ("not measured").

Step 6: Cleaning

Generated values are rounded to the precision used in hospital records (for example, ST depression to 0.1) and checked against medically possible ranges. The ranges were chosen to include every real measured value, so no real patient would be removed.

Step 7: Evaluation protocol

All four methods receive the same data, the same missing-value flags, the same cleaning and the same tests. Each experiment is repeated with five seeds, and methods are compared seed by seed with paired 95% confidence intervals. Table 3.6 lists every measure used; those marked "new" were developed in this project.

Table 3.6: Evaluation measures

Measure	Question it answers	Best	Group
Fidelity (%)	Do single columns look real? (histogram overlap)	Higher	Realism
Detectability (%)	Can a classifier tell fake from real?	50%	Realism
Correlation gap	Do relationships between columns match?	0	Realism
TSTR AUC / AUC ratio	Does a predictor trained on fake data work on real patients?	Higher / 1.0	Usefulness
DCR (%)	Are fake records closer to training patients than to unseen ones?	50%	Privacy
MIA AUC	Can an attacker tell which patients were used for training?	0.5	Privacy
Rate gap (new)	Is each column missing as often as in real data?	0	Missing values
Site gap (new)	Is each column missing as often at each hospital?	0	Missing values
Pattern distance (new)	Do whole patterns of gaps match?	0	Missing values
Rare-pattern MIA (new)	Are patients with unusual gaps easier to identify?	0.5	Privacy of gaps
Unique-pattern leak (new)	Do one-of-a-kind gap patterns of training patients reappear?	0	Privacy of gaps

Step 8: New missing-value measures

A patient's *missingness pattern* is the list of which of their values are blank, together with their hospital. We built the following measures on it.

- The rate gap is the average difference, over columns, in the share of blank values between real and synthetic data.
- The site gap is the same difference computed for each hospital and column, averaged with hospital weights. It catches a model that gets the overall rates right but puts the gaps at the wrong hospital.
- The pattern distance is the total variation distance between the distributions of whole patterns, $\frac{1}{2} \Sigma |P_real(p) - P_synthetic(p)|$. It also captures which values go missing together.
- These scores depend on sample size, so we score the synthetic data on samples the size of the held-out set and score the held-out real patients the same way. We call the second number the real-vs-real reference.

A synthetic score close to it is as good as a second sample of real patients.

- The rare-pattern membership attack is the usual distance-based membership attack, run separately on patients whose pattern is rare (held by at most 2% of patients) and on everyone else.
- The unique-pattern leak looks at patterns held by exactly one patient. It is the share of them that shows up again in the synthetic data when that patient was in training, minus the share when the patient was held out.

Ablation and sensitivity design

To measure what each component contributes, MedTwin was retrained with one component switched off at a time: missing-value flags, zero flag, hospital conditioning and correlation loss. To check that results do not depend on lucky settings, the correlation weight (0.5 and 8 instead of 2) and training length (300 and 1200 instead of 600 epochs) were varied.

3.2) Circuit Layouts / Block Diagrams

Fig. 3.1 shows the complete MedTwin pipeline, from the real data and settings file to the evaluated synthetic data.

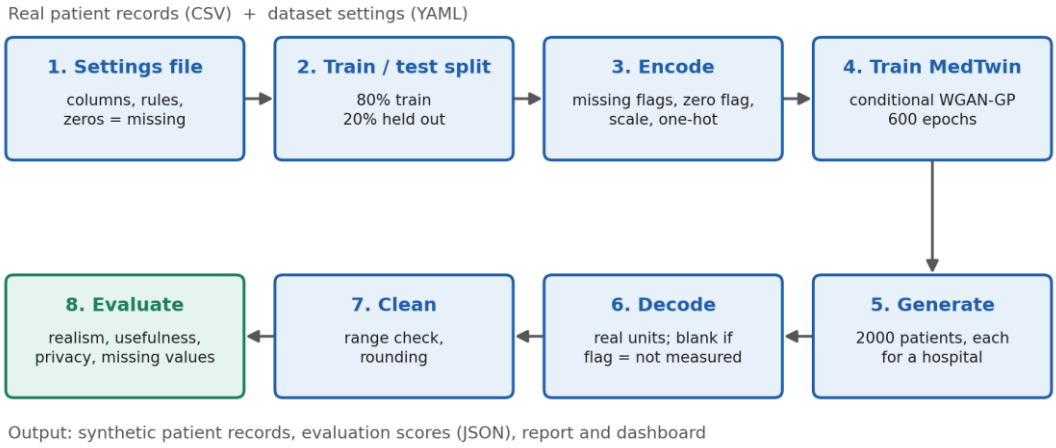


Figure 3.1: Block diagram of the MedTwin pipeline

Fig. 3.2 shows the conditional WGAN-GP. The generator receives noise and the hospital; its outputs are split into sigmoid outputs (numbers and flags) and Gumbel-softmax outputs (categories). The critic receives either a generated or a real record together with the same hospital and outputs a realness score.

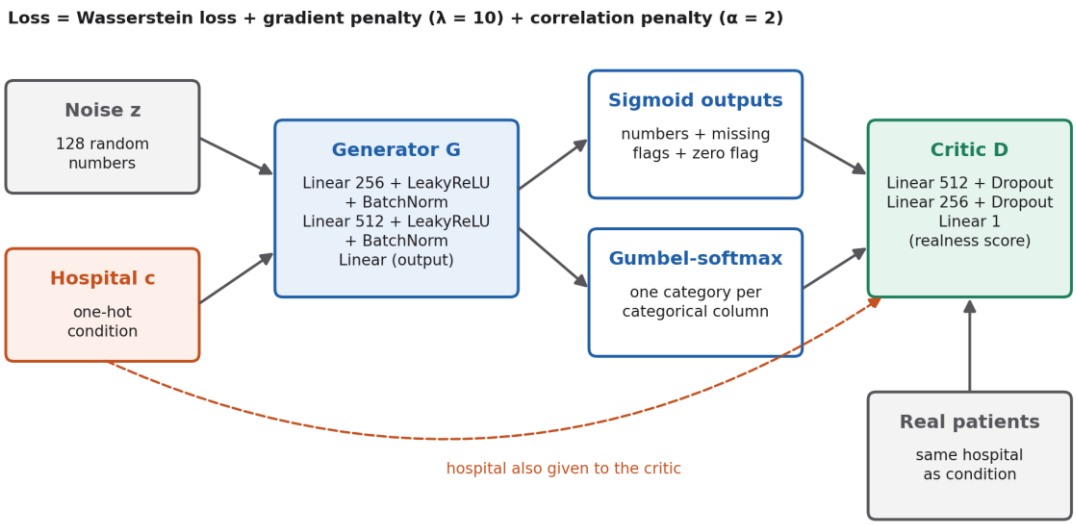


Figure 3.2: Architecture of the MedTwin conditional WGAN-GP

Fig. 3.3 shows the evaluation framework: the held-out patients are never seen by the generator and are used for all usefulness, privacy and missing-value tests.

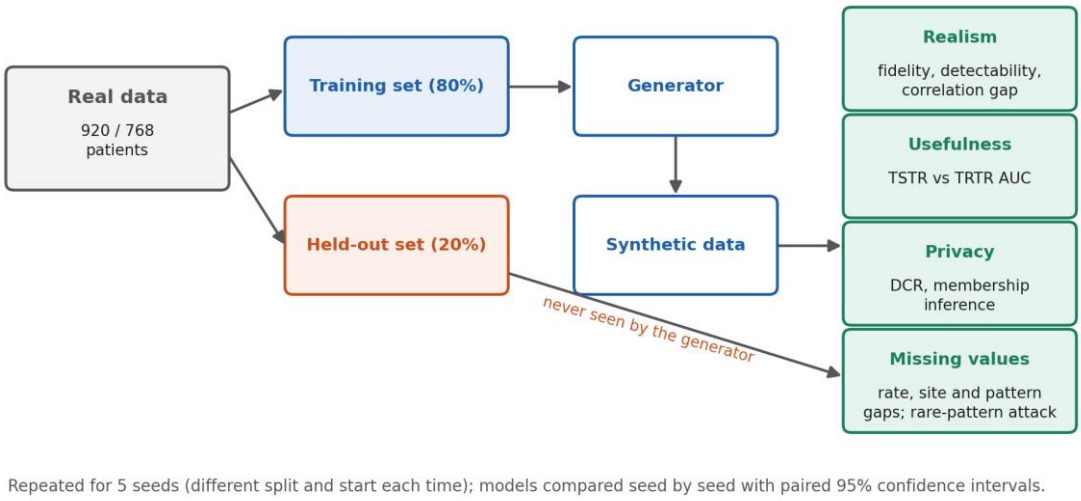


Figure 3.3: Evaluation framework

IMPLEMENTATION

4.1) Modules

The system is organised into small, separately tested modules. Table 4.1 lists the main files and their purpose.

Table 4.1: Software modules

File / folder	Purpose
main.py	Runs the full pipeline once and saves synthetic data and scores
core/config.py	Loads a dataset's settings file (columns, rules, ranges)
core/ingestion.py	Missing-value handling, flags, scaling, encoding
core/generator.py	Generator, critic and WGAN-GP training loop
core/medtwin.py	Train-and-generate steps; ablation and sensitivity switches
core/medical_logic.py	Medical range check and rounding
core/evaluation.py	Usefulness, privacy and correlation tests on held-out patients
core/missingness.py	New missing-value fidelity scores and privacy attacks
research/baselines.py, tabddpm.py	The three comparison methods
research/multi_seed.py	Five-seed comparison with paired statistics
research/ablation.py	Ablation and sensitivity studies
research/visualizer.py, paper_tables.py	Figures and paper tables
app/dashboard.py	Interactive Streamlit dashboard
configs/*.yaml	One settings file per dataset
tests/	31 automated tests

1. Settings module

Everything specific to a dataset lives in a YAML settings file: which columns are numbers or categories, which column is the hospital, which zeros mean "not measured", which columns have a spike at zero, the medically possible ranges and the prediction target. Adding a dataset only needs a new settings file and no code changes. That is how we added the diabetes data.

2. Data ingestion module

This module marks zero-coded values as missing, creates the missing-value and zero flags, fills blanks with placeholder medians, scales numbers to 0–1, one-hot encodes categories (with "not measured" as a category) and prepares

the hospital condition. It also records the layout of the generator's outputs so that the model can be built automatically for any dataset.

3. Generator and training module

This module defines the conditional generator and critic (Table 3.4) and the WGAN-GP training loop, with five critic updates per generator update, the gradient penalty and the correlation loss. During development we found a bug here. A batch with a constant column produced NaN values, and these broke the whole model. Skipping constant columns in the correlation loss fixed it.

4. Generation, cleaning and evaluation modules

After training, synthetic patients are generated for hospitals drawn in real proportions, decoded back to real units, blanked where the flags say "not measured", range-checked and rounded. The evaluation module then computes fidelity, detectability, TSTR utility, DCR, membership inference and the correlation gap against the held-out patients, and the missingness module computes the new missing-value scores and privacy attacks.

5. Baselines and experiment modules

The comparison methods went through exactly the same pipeline (Table 4.2). None of them accepts missing values, so they got the same flag-and-placeholder encoding as MedTwin. The experiment scripts repeat every comparison over five seeds, save each run's results and synthetic data, and can pick up again if stopped. After we added new measurements and re-ran everything, all 400 saved results came out exactly the same, so the experiments are reproducible.

Table 4.2: Methods compared and average training time (heart disease)

Method	Description	Training time
CTGAN [30]	Conditional tabular GAN; 300 epochs, batch 100	1.2 min
TVAE [30]	Variational autoencoder for tables; 300 epochs, batch 100	0.4 min
TabDDPM [31]	Diffusion model for tables (our re-implementation); 8000 steps	3.2 min
MedTwin (ours)	Conditional WGAN-GP with missing-value flags; 600 epochs	6.8 min

4.2) Prototype

A working prototype was built as an interactive web dashboard using Streamlit (Fig. 4.1). It loads the latest synthetic data and shows: the number of synthetic patients, statistical fidelity, an average clinical value and the detectability

score; the held-out evaluation (TSTR utility, utility ratio, DCR and membership-inference AUC); interactive distribution and correlation comparisons between real and synthetic patients; and a data explorer in which blank values are shown as "not measured". Users can filter the synthetic patients by sex and age range and export the filtered records as a CSV file.

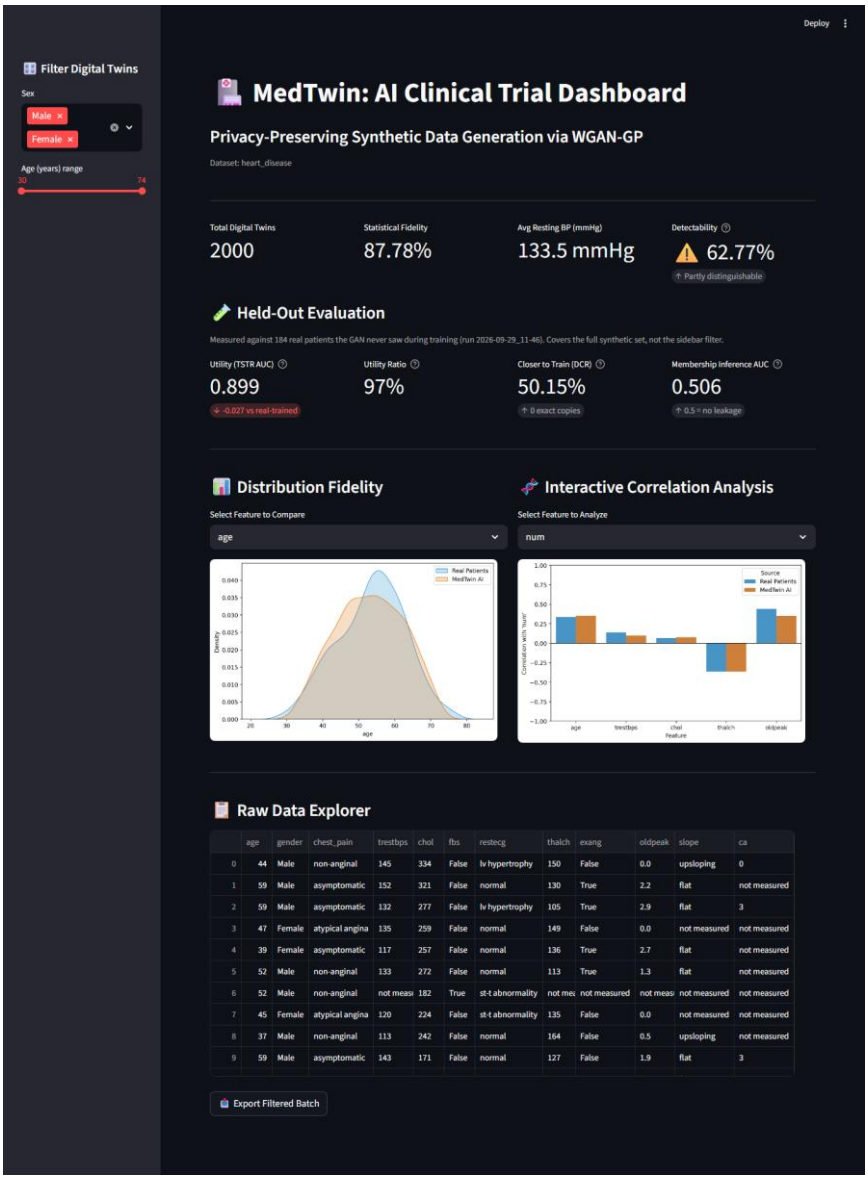


Figure 4.1: MedTwin interactive dashboard (heart disease data)

The pipeline can also be run from the command line: one command trains MedTwin, generates synthetic patients, runs all tests and prints a summary of fidelity, detectability, utility, privacy and missing-value scores. A single option switches to another dataset (Annexure C).

4.3) Testing

The project includes 31 automated tests (pytest) that run in under a minute with very short training. They check, among other things, that: the training

matrix has no blanks and matches the generator's output layout; every generated category is exactly one value; conditioning on a hospital produces patients only for that hospital; all generated values respect the medical ranges; real patients handed in as "synthetic" are caught by the privacy tests; data with every gap filled gets a bad missing-value score; every ablation variant trains and produces a valid report; and results are unchanged when synthetic data is saved to CSV and read back.

Results and Analysis

All results are averages over five repeated experiments ± standard deviation, measured on real patients that the models never saw. Arrows show what is better: ↑ higher, ↓ lower, → closer to the given value. We call a difference *clear* when its paired 95% confidence interval excludes zero.

5.1) Heart Disease: MedTwin Compared with Other Methods

Table 5.1: Heart disease results (mean ± SD over five seeds)

Method	Fidelity % (↑)	Detect. % (→50)	TSTR AUC (↑)	AUC ratio (→1)	DCR % (→50)	MIA AUC (→0.5)
MedTwin (ours)	87.5 ± 0.8	61.5 ± 1.3	0.863 ± 0.023	0.975 ± 0.032	52.9 ± 2.8	0.533 ± 0.030
TVAE	85.6 ± 1.1	79.5 ± 2.8	0.873 ± 0.022	0.987 ± 0.024	50.1 ± 3.4	0.531 ± 0.023
TabDDPM	86.1 ± 1.1	74.9 ± 2.8	0.820 ± 0.030	0.926 ± 0.022	52.7 ± 3.0	0.526 ± 0.033
CTGAN	74.2 ± 4.6	92.8 ± 1.6	0.696 ± 0.069	0.787 ± 0.082	51.1 ± 3.4	0.507 ± 0.020

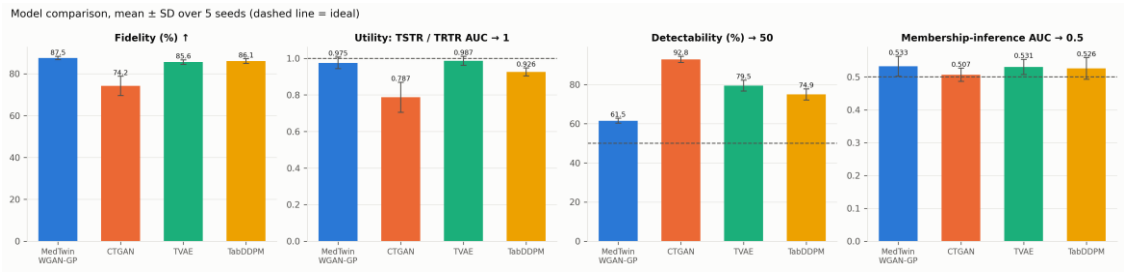


Figure 5.1: Heart disease comparison of the four methods (mean ± SD over five seeds; dashed lines show the ideal value)

MedTwin has the highest fidelity (87.5%) and the lowest detectability (61.5%). The other methods reach 74.9% (TabDDPM), 79.5% (TVAE) and 92.8% (CTGAN), and both differences are clear against each of them. MedTwin's AUC ratio is 0.975, level with TVAE (0.987) and clearly better than CTGAN (0.787). DCR and membership-inference AUC are close to their ideal values for all four methods, and none of them produced an exact copy of a training patient.

Generating each patient for a hospital, and generating the gaps instead of filling them, makes MedTwin's records much harder for a classifier to pick out, and the data stays just as useful. Detectability (61.5%) is still above the ideal 50%, so there is room to improve.

5.2) Diabetes: The Same Picture

Table 5.2: Pima diabetes results (mean ± SD over five seeds)

Method	Fidelity % (↑)	Detect. % (→50)	TSTR AUC (↑)	AUC ratio (→1)	DCR % (→50)	MIA AUC (→0.5)
MedTwin (ours)	90.9 ± 0.8	64.0 ± 2.6	0.801 ± 0.030	0.983 ± 0.041	51.8 ± 1.8	0.494 ± 0.021
TVAE	88.0 ± 1.7	70.0 ± 1.4	0.806 ± 0.013	0.989 ± 0.028	54.0 ± 2.4	0.518 ± 0.024
TabDDPM	85.4 ± 1.3	55.9 ± 2.6	0.781 ± 0.050	0.957 ± 0.026	60.8 ± 4.9	0.803 ± 0.028
CTGAN	77.8 ± 3.3	86.3 ± 2.7	0.568 ± 0.062	0.697 ± 0.075	49.7 ± 7.7	0.505 ± 0.028

- MedTwin again has the best fidelity (90.9%) and ties for usefulness (AUC ratio 0.983).
- TabDDPM is the hardest to detect (55.9%), but it memorised its training patients. 60.8% of its records are closer to training patients than to unseen ones, and the membership attack reaches an AUC of 0.803. MedTwin shows no such leak (0.494). This agrees with recent findings that tabular diffusion models memorise [37], and it is a good example of why detectability should not be read as privacy.

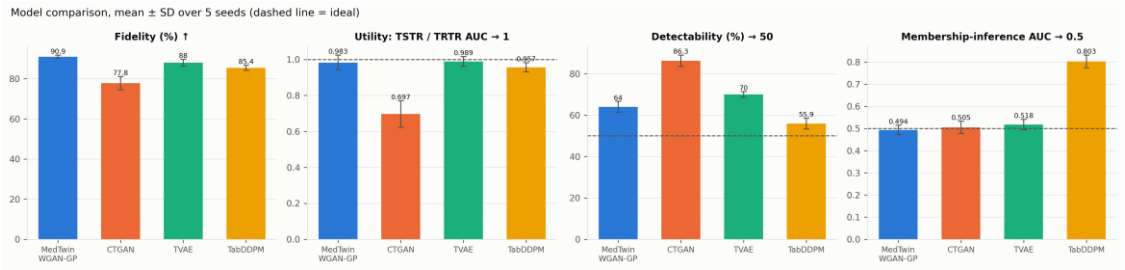


Figure 5.2: Pima diabetes comparison of the four methods

5.3) Are the Missing Values Realistic?

Table 5.3: Missing-value fidelity (mean ± SD; lower is better)

Data	Rate gap	Site gap	Pattern distance
Heart disease			
Two real samples (reference)	0.020	0.024	0.152
MedTwin (ours)	0.021 ± 0.002	0.022 ± 0.001	0.174 ± 0.006
TVAE	0.028 ± 0.005	0.035 ± 0.002	0.209 ± 0.009
TabDDPM	0.032 ± 0.005	0.101 ± 0.004	0.487 ± 0.028
CTGAN	0.059 ± 0.005	0.131 ± 0.006	0.683 ± 0.014
MedTwin without missing-value flags	0.118 ± 0.002	0.121 ± 0.002	0.693 ± 0.009
MedTwin without hospital conditioning	0.019 ± 0.002	0.028 ± 0.001	0.223 ± 0.012

Data	Rate gap	Site gap	Pattern distance
Diabetes (one clinic, so no site gap)			
Two real samples (reference)	0.030	–	0.085
MedTwin (ours)	0.019 ± 0.002	–	0.063 ± 0.006
TVAE	0.031 ± 0.005	–	0.118 ± 0.022
TabDDPM	0.029 ± 0.010	–	0.111 ± 0.013
CTGAN	0.089 ± 0.033	–	0.299 ± 0.032
MedTwin without missing-value flags	0.167 ± 0.006	–	0.486 ± 0.018

- MedTwin's pattern distance on heart disease (0.174) is close to the gap between two real samples (0.152) and clearly better than every other method in all five seeds (TVAE 0.209, TabDDPM 0.487, CTGAN 0.683).
- Only MedTwin puts the gaps at the right hospitals. Its site gap (0.022) matches the real data (0.024). The other methods got the same flags and their overall missing rates are close to real, but they do not know which hospital a gap belongs to, so they put gaps in the wrong places (site gaps 0.035–0.131). A per-column check alone would not show this.
- Without the missing-value flags the pattern distance jumps to 0.693; without hospital conditioning the overall rates stay right but the site gap (0.028) and pattern distance (0.223) clearly worsen.
- On diabetes, MedTwin again has the lowest pattern distance (0.063; other methods 0.111–0.299).

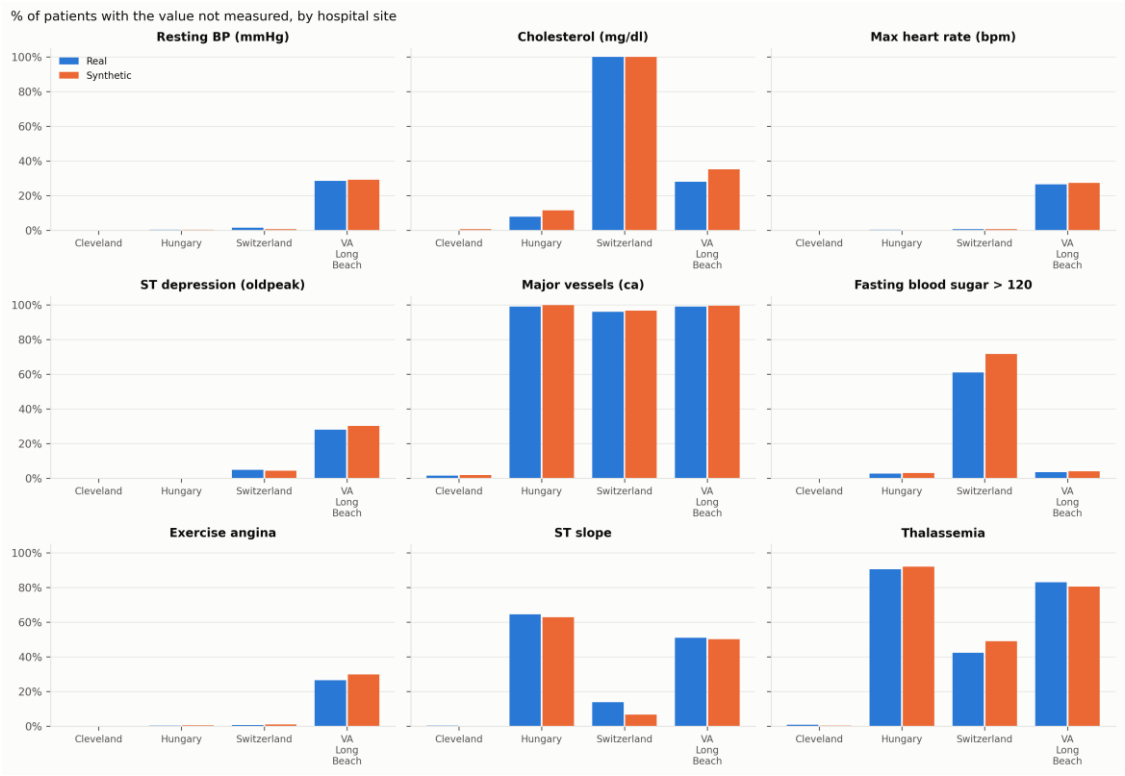


Figure 5.3: Share of heart disease patients with each value not measured, per hospital: real (blue) and MedTwin (orange)

Fig. 5.3 shows that MedTwin reproduces the hospital-specific structure: cholesterol missing for every Swiss patient, vessel counts missing almost everywhere except Cleveland, and thalassemia missing for most Hungarian and VA patients. The largest differences are for fasting blood sugar in Switzerland (61% real, 72% synthetic) and ST slope in Switzerland (14% real, 7% synthetic).

5.4) Does Keeping Missing Values Cost Privacy?

Table 5.4: Missing-value privacy audit, heart disease (mean ± SD; 0.5 or 0 is ideal)

Variant	MIA rare patterns	MIA common patterns	MIA all patients	Unique leak (points)
Full MedTwin	0.575 ± 0.046	0.542 ± 0.042	0.532 ± 0.030	-4.7 ± 25.3
MedTwin without missing-value flags	0.531 ± 0.049	0.516 ± 0.016	0.515 ± 0.027	1.9 ± 21.8

On heart disease about 64 of the 368 attacked patients per test have a rare missingness pattern. For them, the membership attack reaches an AUC of 0.575 against MedTwin, compared with 0.531 when MedTwin is trained without the missing-value flags, and it is higher in all five seeds. For patients with common patterns, and over all patients, the difference is not clear. Patterns unique to one training patient do not reappear more often than those of unseen patients, so the model is not copying patterns.

So keeping realistic gaps makes the data more realistic and leaves overall privacy where it was, but patients with *unusual* gaps become slightly easier to single out. The effect is small. Still, these are the patients a data owner would worry about most. On diabetes only about 8 patients per test have rare patterns, which is too few to draw a conclusion.

5.5) What Each Component Contributes (Ablation)

Table 5.5: Ablation study (mean ± SD; * = clear difference from the full model)

Variant	Detect. % (→50)	Corr. gap (↓)	AUC ratio (→1)	MIA AUC (→0.5)
Heart disease				
Full MedTwin	62.2 ± 1.7	0.041 ± 0.007	0.974 ± 0.028	0.532 ± 0.030
without missing-value flags	84.2 ± 1.2 *	0.071 ± 0.007 *	0.989 ± 0.025	0.515 ± 0.027
without zero flag	74.9 ± 0.9 *	–	0.978 ± 0.032	0.539 ± 0.038
without hospital conditioning	65.6 ± 1.7 *	0.046 ± 0.005	0.975 ± 0.038	0.527 ± 0.025
without correlation loss	65.2 ± 2.4	0.075 ± 0.018 *	0.982 ± 0.011	0.525 ± 0.032
Diabetes				
Full MedTwin	64.0 ± 2.6	0.034 ± 0.007	0.983 ± 0.041	0.494 ± 0.021
without missing-value flags	81.5 ± 1.6 *	0.042 ± 0.004 *	0.993 ± 0.032	0.524 ± 0.055
without correlation loss	63.8 ± 2.9	0.063 ± 0.008 *	0.964 ± 0.032	0.508 ± 0.029

- The missing-value flags matter most. Without them, detectability rises from 62.2% to 84.2% (heart) and from 64.0% to 81.5% (diabetes). Filling gaps with typical values makes synthetic patients easy to spot, which matches the "impute-then-generate" bias reported for MissDiff [16].
- The zero flag and hospital conditioning also clearly help realism (detectability 74.9% and 65.6% without them).
- The correlation loss does not change realism or usefulness. Without it, though, the correlation gap between numeric columns nearly doubles (0.041 → 0.075 heart; 0.034 → 0.063 diabetes).
- No component changes usefulness or overall privacy. The medical range check never removed a patient, so it is only a safety net.

5.6) Sensitivity to Settings

Table 5.6: Sensitivity of MedTwin to its main settings, heart disease (* = clear difference from the default)

Setting	Fidelity %	Detect. %	AUC ratio	DCR %	MIA AUC
Default (weight 2, 600 epochs)	87.2 ± 0.7	62.2 ± 1.7	0.974 ± 0.028	53.3 ± 2.0	0.532 ± 0.030
Correlation weight 0.5	87.0 ± 0.8	62.8 ± 2.8	0.983 ± 0.021	53.3 ± 2.1	0.531 ± 0.023

Setting	Fidelity %	Detect. %	AUC ratio	DCR %	MIA AUC
Correlation weight 8	87.9 ± 0.5	62.8 ± 2.3	0.989 ± 0.031	53.9 ± 2.6	0.525 ± 0.032
300 epochs	87.0 ± 0.5	63.6 ± 2.8	0.961 ± 0.034	52.2 ± 2.5	0.524 ± 0.035
1200 epochs	87.2 ± 0.5	63.0 ± 2.2	0.984 ± 0.017	55.3 ± 2.6 *	0.548 ± 0.024 *

Changing the correlation weight or training for 300 epochs makes no clear difference. Training twice as long (1200 epochs) gives the same realism and usefulness but starts to memorise: DCR rises from 53.3% to 55.3% and MIA AUC from 0.532 to 0.548. So the results do not rest on lucky settings, and 600 epochs is a sensible place to stop.

5.7) Quality of the Generated Values

Figs. 5.4 and 5.5 compare the distributions of the number columns, and the correlations between them, for real and synthetic heart disease patients. The shapes and the main relationships, such as maximum heart rate falling with age, are kept.

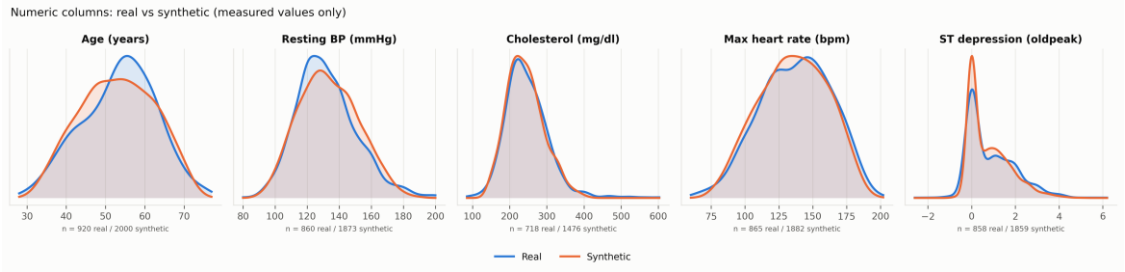


Figure 5.4: Distributions of the number columns: real (blue) vs MedTwin (orange), measured values only

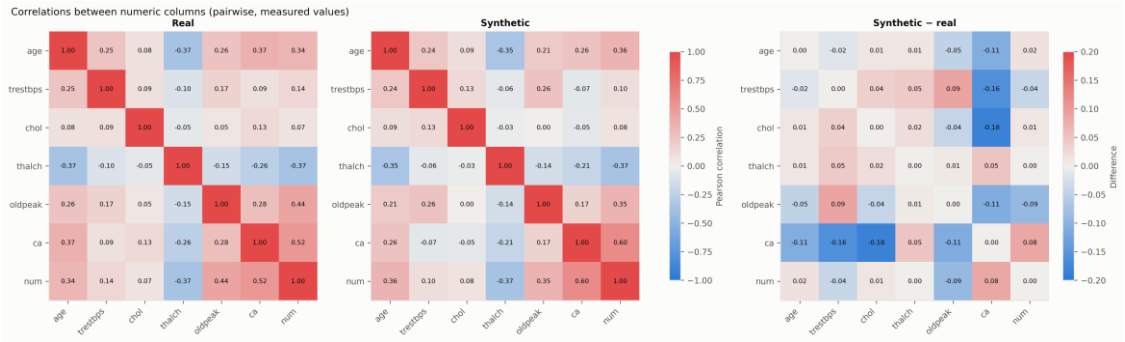


Figure 5.5: Correlations between number columns: real, MedTwin and their difference

5.8) Challenges Faced and Learnings

- Training sometimes broke down. Batches that happened to contain a constant column produced NaN values in the correlation loss, and these corrupted the whole model. Skipping constant columns fixed it.

- Some missing values were hidden as zeros, for example Swiss cholesterol. They distorted our early results until the settings file marked them as missing.
- Early versions tested on patients the model had already seen, and reported a "privacy" score that really measured realism. Holding out patients and keeping detectability separate from privacy fixed both problems.
- Single runs gave misleading differences between methods. Repeating each run with five seeds and using paired confidence intervals showed which differences were real.
- The full comparison takes about an hour. Because each run's results and synthetic data were saved, experiments could resume after a stop, and new measures could be added without retraining.

Conclusions

We built and tested MedTwin, a model that generates synthetic patient records and keeps the real pattern of missing values. Our main conclusions are below.

1. Realistic synthetic patients, including their gaps

- **Outcome:** MedTwin produced the most realistic records of the four methods tested (detectability 61.5% on heart disease against 74.9–92.8%) and the highest fidelity on both datasets.
- **Significance:** keeping "not measured" in the data instead of filling it in is the main reason the records look real. In the ablation study, the missing-value flags made the biggest difference.

2. Useful and private

- **Outcome:** a predictor trained on MedTwin's data keeps 97.5% (heart) and 98.3% (diabetes) of real-data performance, and privacy attacks found no memorisation.
- **Significance:** MedTwin became more realistic without losing usefulness or privacy. TabDDPM, by contrast, memorised its training patients on the diabetes data.

3. New ways to measure missing values

- **Outcome:** the new missing-value scores showed that MedTwin's gap patterns are almost as close to the real data as a second real sample (0.174 vs 0.152), and that only MedTwin places gaps at the right hospitals.
- **Significance:** these scores show differences that per-column checks miss, and they work for any synthetic-data generator.

4. A privacy cost of keeping gaps

- **Outcome:** keeping realistic gaps raises the membership attack on patients with rare patterns from 0.531 to 0.575, while overall privacy is unchanged.
- **Significance:** anyone releasing such data should know about this small cost for patients with unusual gaps. Reducing it is an obvious next step.

The work has limits. Both datasets are small (under 1000 patients), and detectability is still above the ideal 50%. Privacy was tested with attacks and is not guaranteed mathematically [27]. The audit of gaps needs enough patients with rare patterns, so it was inconclusive on diabetes. And the newest missing-value-aware generators were not part of the comparison.

Future Scope of Work

Several directions follow from this work.

1. Comparison with missing-value-aware generators

- **Objective:** Compare MedTwin directly with the closest previous methods, such as MissDiff and the missing-pattern-preserving approach of Wang et al.
- **Applications:**
  - Benchmark all methods with the new missing-value scores.
  - Identify which design choices matter most for realistic gaps.
- **Significance:** Shows how MedTwin compares with the newest methods.

2. Protecting patients with unusual gaps

- **Objective:** Reduce the privacy cost found for patients with rare missingness patterns.
- **Applications:**
  - Merge or coarsen rare patterns before release.
  - Train with differential privacy and measure the realism it costs.
- **Significance:** Lets realistic gaps be kept while protecting the patients most at risk.

3. Generating data for new hospitals

- **Objective:** Use hospital conditioning to generate patients for a hospital the model has never seen.
- **Applications:**
  - Train on three hospitals and generate for the fourth.
  - Test whether models trained on these patients work at the new hospital.
- **Significance:** Would help hospitals share data and build models together.

4. Larger and time-based records

- **Objective:** Extend MedTwin to large hospital databases and records collected over time.
- **Applications:**
  - Patients with many visits and changing measurements.
  - Thousands of patients and many more columns.
- **Significance:** Brings the work closer to real hospital databases.

References

[1] A. Goncalves, P. Ray, B. Soper, J. Stevens, L. Coyle, and A. P. Sales, "Generation and evaluation of synthetic patient data," BMC Medical Research Methodology, vol. 20, art. no. 108, 2020, doi: 10.1186/s12874-020-00977-1.

[2] C. Yan, Y. Yan, Z. Wan, Z. Zhang, L. Omberg, J. Guinney, S. D. Mooney, and B. A. Malin, "A multifaceted benchmarking of synthetic electronic health record generation models," Nature Communications, vol. 13, art. no. 7609, 2022, doi: 10.1038/s41467-022-35295-1.

[3] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in Advances in Neural Information Processing Systems 27, 2014, pp. 2672–2680.

[4] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in Proc. 34th International Conference on Machine Learning, PMLR, vol. 70, 2017, pp. 214–223.

[5] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of Wasserstein GANs," in Advances in Neural Information Processing Systems 30, 2017.

[6] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart Disease," UCI Machine Learning Repository, 1989, doi: 10.24432/C52P4X.

[7] R. Detrano, A. Janosi, W. Steinbrunn, M. Pfisterer, J.-J. Schmid, S. Sandhu, K. H. Guppy, S. Lee, and V. Froelicher, "International application of a new probability algorithm for the diagnosis of coronary artery disease," The American Journal of Cardiology, vol. 64, no. 5, pp. 304–310, 1989, doi: 10.1016/0002-9149(89)90524-9.

[8] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in Proc. Annual Symposium on Computer Application in Medical Care, 1988, pp. 261–265.

[9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32, 2019.

[10] F. Pedregosa, G. Varoquaux, A. Gramfort et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] W. A. Wan Abu Bakar, N. L. N. Josdi, M. Man, and E. A. Kadir, "Evaluating CTGAN-generated synthetic data for heart disease prediction: Fidelity, predictive utility, and feature preservation," International Journal of Advanced Computer Science and Applications, vol. 16, no. 12, pp. 1000–1009, 2025, doi: 10.14569/IJACSA.2025.0161296.

[12] P. Subedi, S. R. Divakarla, M. Emani, S. Kochuvila, and R. Sunitha, "Synthetic data for health risk prediction: An evaluation using machine learning," in Proc. 5th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS), IEEE, 2025, doi: 10.1109/ICUIS67429.2025.11380688.

[13] M. Alqulaity and P. Yang, "Enhanced conditional GAN for high-quality synthetic tabular data generation in mobile-based cardiovascular healthcare," Sensors, vol. 24, no. 23, art. no. 7673, 2024, doi: 10.3390/s24237673.

[14] I. Lazzaro, M. Milano, A. Tucker, and M. Cannataro, "Assessing the impact of synthetic data generated by Bayesian networks on heart disease prediction," Journal of Computational Science, vol. 99, art. no. 102940, 2026, doi: 10.1016/j.jocs.2026.102940.

[15] S. Bing, A. Dittadi, S. Bauer, and P. Schwab, "Conditional generation of medical time series for extrapolation to underrepresented populations," arXiv:2201.08186, 2022.
[16] Y. Ouyang, L. Xie, C. Li, and G. Cheng, "MissDiff: Training diffusion models on tabular data with missing values," ICML 2023 Workshop on Structured Probabilistic Inference and Generative Modeling, arXiv:2307.00467, 2023.
[17] X. Wang, H. Asif, S. Gupta, and J. Vaidya, "Data synthesis reinvented: Preserving missing patterns for enhanced analysis," IEEE Transactions on Knowledge and Data Engineering, vol. 37, no. 7, pp. 3962–3975, 2025.
[18] W. Wang, P. Tang, J. Lou, Y. Shao, L. Waller, Y. Ko, and L. Xiong, "IGAMT: Privacy-preserving electronic health record synthesization with heterogeneity and irregularity," in Proc. AAAI Conference on Artificial Intelligence, vol. 38, no. 14, 2024, pp. 15634–15643, doi: 10.1609/aaai.v38i14.29491.
[19] M. Hernandez, P. A. Osorio-Marulanda, M. Catalina, L. Loinaz, G. Epelde, and N. Aginako, "Comprehensive evaluation framework for synthetic tabular data in health: Fidelity, utility and privacy analysis of generative models with and without privacy guarantees," Frontiers in Digital Health, vol. 7, art. no. 1576290, 2025, doi: 10.3389/fdgth.2025.1576290.
[20] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-softmax," in Proc. International Conference on Learning Representations (ICLR), 2017.
[21] J. Yoon, J. Jordon, and M. van der Schaar, "GAIN: Missing data imputation using generative adversarial nets," in Proc. 35th International Conference on Machine Learning, PMLR, vol. 80, 2018, pp. 5689–5698.
[22] S. C.-X. Li, B. Jiang, and B. M. Marlin, "MisGAN: Learning from incomplete data with generative adversarial networks," in Proc. International Conference on Learning Representations (ICLR), 2019.
[23] A. Alaa, B. van Breugel, E. Saveliev, and M. van der Schaar, "How faithful is your synthetic data? Sample-level metrics for evaluating and auditing generative models," in Proc. 39th International Conference on Machine Learning, PMLR, vol. 162, 2022, pp. 290–306.
[24] C. Esteban, S. L. Hyland, and G. Rätsch, "Real-valued (medical) time series generation with recurrent conditional GANs," arXiv:1706.02633, 2017.
[25] J. Hayes, L. Melis, G. Danezis, and E. De Cristofaro, "LOGAN: Membership inference attacks against generative models," Proceedings on Privacy Enhancing Technologies, vol. 2019, no. 1, pp. 133–152, 2019, doi: 10.2478/popets-2019-0008.
[26] D. Chen, N. Yu, Y. Zhang, and M. Fritz, "GAN-Leaks: A taxonomy of membership inference attacks against generative models," in Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2020, doi: 10.1145/3372297.3417238.
[27] T. Stadler, B. Oprisanu, and C. Troncoso, "Synthetic data – anonymisation groundhog day," in Proc. 31st USENIX Security Symposium, 2022, pp. 1451–1468.
[28] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd International Conference on Learning Representations (ICLR), 2015.
[30] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in Advances in Neural Information Processing Systems 32, 2019.

[31] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in Proc. 40th International Conference on Machine Learning, PMLR, vol. 202, 2023, pp. 17564–17579.
[32] E. Choi, S. Biswal, B. Malin, J. Duke, W. F. Stewart, and J. Sun, "Generating multi-label discrete patient records using generative adversarial networks," in Proc. 2nd Machine Learning for Healthcare Conference, PMLR, vol. 68, 2017, pp. 286–305.
[33] Z. Zhao, A. Kunar, R. Birke, and L. Y. Chen, "CTAB-GAN: Effective table data synthesizing," in Proc. 13th Asian Conference on Machine Learning, PMLR, vol. 157, 2021, pp. 97–112.
[34] H. Zhang, J. Zhang, B. Srinivasan, Z. Shen, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in Proc. International Conference on Learning Representations (ICLR), 2024.
[35] J. Yoon, M. Mizrahi, N. F. Ghalaty, T. Jarvinen, A. S. Ravi, P. Brune, F. Kong, D. Anderson, G. Lee, A. Meir, F. Bandukwala, E. Kanal, S. Ö. Arık, and T. Pfister, "EHR-Safe: Generating high-fidelity and privacy-preserving synthetic electronic health records," npj Digital Medicine, vol. 6, art. no. 141, 2023, doi: 10.1038/s41746-023-00888-7.
[36] Z. Qian, R. Davis, and M. van der Schaar, "Synthcity: A benchmark framework for diverse use cases of tabular synthetic data," in Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track, 2023.
[37] Z. Fang, Z. Jiang, H. Chen, X. Li, and J. Li, "Understanding and mitigating memorization in diffusion models for tabular data," in Proc. 42nd International Conference on Machine Learning, PMLR, vol. 267, 2025, pp. 15976–16005.
[38] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," ACM SIGKDD Explorations Newsletter, vol. 15, no. 2, pp. 49–60, 2014, doi: 10.1145/2641190.2641198.

Annexures

Annexure A: Sample synthetic patients

The first eight synthetic heart disease patients generated by MedTwin (seed 0). Blank values are shown as "not measured". None of these records belongs to a real patient.

Table A.1: Sample of MedTwin's synthetic heart disease patients

age	gender	chest_pain	trestbps	chol	thalch	oldpeak	ca	thal	num	origin_study
58	Female	non-anginal	110	259	120	0.5	not measured	not measured	0	Hungary
60	Male	non-anginal	182	not measured	85	0	not measured	reversable defect	3	Switzerland
45	Male	asymptomatic	145	239	145	0	not measured	not measured	0	Hungary
42	Male	atypical angina	118	266	149	0	not measured	not measured	0	Hungary
45	Male	non-anginal	127	168	142	1.6	not measured	not measured	0	Hungary
51	Male	asymptomatic	143	317	149	2.1	not measured	not measured	2	Hungary
57	Male	asymptomatic	158	235	103	2.7	not measured	not measured	0	Hungary
61	Male	asymptomatic	172	196	116	not measured	not measured	not measured	1	VA Long Beach

Annexure B: Code snippets

B.1 Creating the missing-value and zero flags (core/ingestion.py):

```
flags = pd.DataFrame(
    {missing_flag(c): df[c].isna().astype(float) for c in self.flagged_cols},
    index=df.index,
)
for col in self.zero_cols:
    flags[zero_flag(col)] = (df[col] == 0).astype(float)
```

B.2 Generator output layer (core/generator.py): sigmoid for numbers and flags, Gumbel-softmax for each category.

```
def forward(self, z, cond, hard=False):
    logits = self.model(torch.cat([z, cond], dim=1))
    outputs, start = [], 0
    for kind, width in self.output_info:
        segment = logits[:, start:start + width]
        if kind == "sigmoid":
            outputs.append(torch.sigmoid(segment))
        else:
            outputs.append(F.gumbel_softmax(segment, tau=GUMBEL_TAU, hard=hard))
        start += width
    return torch.cat(outputs, dim=1)
```

B.3 Pattern distance between real and synthetic gaps

(core/missingness.py):

```
p_ref = patterns(reference, cols).value_counts(normalize=True)
p_oth = patterns(other, cols).value_counts(normalize=True)
union = p_ref.index.union(p_oth.index)
scores["pattern_tvd"] = float(
    0.5 * (p_ref.reindex(union, fill_value=0)
           - p_oth.reindex(union, fill_value=0)).abs().sum()
)
```

Annexure C: How to run the project

Task	Command
Install	pip install -r requirements.txt
Run the pipeline (heart disease)	python main.py
Run on diabetes	python main.py --config configs/diabetes.yaml
Dashboard	streamlit run app/dashboard.py
5-seed comparison	python research/multi_seed.py
Ablation study	python research/ablation.py
Sensitivity study	python research/ablation.py --study sensitivity
Figures	python research/visualizer.py --benchmark
Automated tests	python -m pytest tests -q

Annexure D: Project timeline

An early version of the project (April 2026) used 9 columns, filled missing values and tested on patients the model had seen. It was rebuilt to handle all columns with missing values and hospitals, to test on unseen patients and to compare against other methods. The main recorded steps since then are:

Date	Step
2026-09-29	Generated all 15 columns with "not measured" handling and hospital conditioning
2026-09-29	Added the oldpeak zero flag, 5-seed comparison and paper figures
2026-09-29	Moved dataset settings into configuration files; added the Pima diabetes dataset
2026-10-15	Added ablation switches and the ablation and sensitivity studies
2026-10-15	Separated detectability from privacy; added the correlation-gap measure
2026-10-15	Added the missing-value fidelity scores and privacy audit
2026-10-16	Verified all references; wrote and compiled the research paper